

PA2: Priority-based Scheduler for xv6

Repository: <https://github.com/dgarciaquezad/xv6-riscv>

Branch: pa2

Code commit: feab252

Test configuration: one CPU

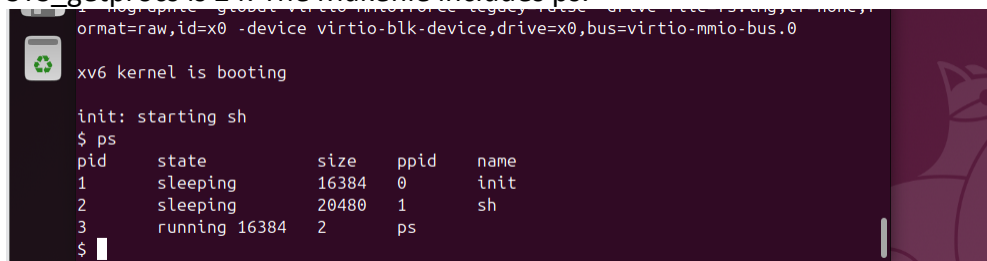
Task 1. (15 pts) Implement the `getprocs()` system call and test it using the provided `ps` command.

I added struct `pstat` in `kernel/pstat.h` and moved enum `procstate` into that shared header so the kernel and user programs use the same state values. I kept the PA1 struct `rusage` definition. `kernel/proc.h` includes `pstat.h` instead of defining `procstate` again.

The syscall handler obtains the user array address and calls `kgetprocs()`. This function allocates a temporary kernel page, scans processes whose state is not `UNUSED`, and records their PID, state, memory size, parent PID, and name. It takes `wait_lock` before each process lock when reading parent information. After releasing the locks, `copyout()` transfers the records into the caller's array. The function frees the page and returns the number of records, or `-1` on failure. The caller reserves space for `NPROC` records.

Files changed: `kernel/pstat.h`, `kernel/proc.h`, `kernel/proc.c`, `kernel/syscall.h`, `kernel/syscall.c`, `kernel/sysproc.c`, `kernel/defs.h`, `user/user.h`, `user/usys.pl`, `user/ps.c`, and `Makefile`.

`SYS_getprocs` is 24. The `Makefile` includes `ps`.



```
format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
init: starting sh
$ ps
pid  state      size  ppid  name
1    sleeping  16384  0     init
2    sleeping  20480  1     sh
3    running  16384  2     ps
$
```

Task 2. (15 pts) Modify the provided `ps` command to print the priority of each process.

I added an integer priority field to struct `proc` and initialized it to 0 in `allocproc()`. `kfork()` copies the parent's priority into the child. Because `init` uses `allocproc()`, its initial priority is also 0.

I registered `getpriority()` and `setpriority()` in the syscall table and user syscall stubs. Their syscall numbers are 25 and 26. `getpriority()` reads the caller's base priority under its process lock. `setpriority()` accepts values from 0 through 49, updates only the caller, and returns 0. An invalid value returns `-1` without changing the priority.

I added priority to struct `pstat`, copied it in `kgetprocs()`, and printed it in `user/ps.c`. Files changed: `kernel/proc.h`, `kernel/proc.c`, `kernel/pstat.h`, `kernel/syscall.h`, `kernel/syscall.c`, `kernel/sysproc.c`, `user/user.h`, `user/usys.pl`, `user/ps.c`, `user/priotest.c`, and `Makefile`.

```
xv6 kernel is booting

init: starting sh
$ priotest
Initial priority: 0
PASS: priority 0 accepted
PASS: priority 49 accepted
PASS: priority changed to 10
PASS: priority -1 rejected without changing priority
PASS: priority 50 rejected without changing priority
PASS: fork succeeded
PASS: fork succeeded
PASS: child inherited priority 10
PASS: child changed its priority to 20
ps should show parent at 10 and child at 20:
pid  state      size  priority  ppid  name
1    sleeping    16384  0         0      init
2    sleeping    20480  0         1      sh
3    sleeping    16384  10        2      priotest
4    running     16384  20        3      ps
PASS: child and ps finished successfully
PASS: parent priority remained 10
All priority tests passed.
$
```

priotest checked accepted values 0 and 49, a change to 10, rejected values -1 and 50, and inheritance across fork(). The child inherited 10, changed itself to 20, and executed ps. The parent stayed at 10. The test ended with All priority tests passed.

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

I added an unsigned readytime field to struct proc and struct pstat. allocproc() clears it initially. Whenever a process becomes RUNNABLE, the code sets readytime to the current ticks value immediately before changing the state. The five updated locations in kernel/proc.c are userinit(), kfork(), yield(), wakeup(), and kkill(). These cover initial readiness, a new child, yielding, ordinary wakeups, and waking a killed sleeping process. kgetprocs() copies readytime into its snapshot.

user/ps.c calls uptime() after collecting the snapshot. For a RUNNABLE record, it displays current time minus readytime in ticks. It leaves age blank for other states. The value describes the current period of waiting to run, rather than the process's lifetime or accumulated waiting time. Files changed: kernel/proc.h, kernel/pstat.h, kernel/proc.c, and user/ps.c. The implementation reads ticks while holding the process lock in this one-CPU configuration. It does not take tickslock in wakeup(), which can already be called by code holding tickslock.

```
$ ps
pid  state      size  age  priority  ppid  name
1    sleeping    16384  0     0         0      init
2    sleeping    20480  0     0         1      sh
9    running     16384  0     0         2      ps
$ matmul 100 &
matmul 100 &
ps
ps$ $ pid      state      size  age  priority  ppid  name
1    sleeping    16384  0     0         0      init
2    sleeping    20480  0     0         1      sh
14   running     16384  0     0         2      ps
11   runnable    65536  1     0         1      matmul
13   runnable    65536  0     0         1      matmul
$ Time: 15 ticks, checksum: 1247680
Time: 13 ticks, checksum: 1247680
```

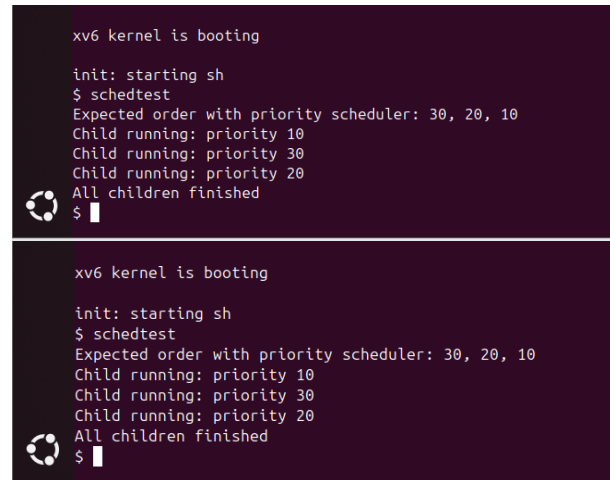
Task 4. (15 pts) Implement a priority-based scheduler.

Files changed: kernel/param.h defines SCHED_RR, SCHED_PRIORITY, and SCHED_POLICY to select a scheduler at compile time. kernel/proc.c keeps the original Round Robin path in scheduler() and adds the strict priority path. Each build was started with make qemu CPUS=1.

The priority path examines RUNNABLE processes and selects the largest numeric base priority. A rotating starting index handles ties. The selected process runs through the normal context switch while its lock is held. I preserved the existing interrupt handling, including mycpu()->intena = 0 after returning to the scheduler.

user/pexec.c validates a priority from 0 to 49, calls setpriority(), and executes the requested command. user/schedtest.c creates children with priorities 10, 30, and 20. Pipes ensure all three are ready before the parent releases them. Makefile registers both programs.

I learned that synchronization is needed to distinguish scheduler policy from fork timing. A difficulty was preserving the original scheduler while adding another selection path. Compile-time branches and testing both policies helped verify the change. The printed expected priority order remains in schedtest under Round Robin, but its observed order is intentionally different



```
xv6 kernel is booting

init: starting sh
$ schedtest
Expected order with priority scheduler: 30, 20, 10
Child running: priority 10
Child running: priority 30
Child running: priority 20
All children finished
$
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

I added SCHED_AGING in kernel/param.h and extended the priority path in scheduler(). Base priorities remain 0 to 49. For each RUNNABLE process, effective priority is its base priority plus the ticks spent waiting since readytime, capped at 99. The calculation does not overwrite the base priority. The code applies the cap before converting a large age to int: if age is at least 99 minus the base priority, effective priority becomes 99; otherwise it adds the age. Yielding or awakening refreshes readytime, so aging starts again for that ready interval. The rotating scan handles equal effective priorities.

Test method

user/agingtest.c creates children at priorities 0 and 49. Readiness and release pipes coordinate their start. The low-priority child performs 3 chunks; the high-priority child performs 1000. Each chunk executes one million updates to a volatile unsigned value so the CPU work is not removed as unused. A results pipe returns the measurements, and the parent waits for both children. Response is measured from the common workload release tick to the child's first work after reading the release message. Turnaround is measured from that release tick to the end of the CPU loop, before sending the result and exiting. These are workload response and completion times, not measurements from fork() to first dispatch or exit. Each average is over the two jobs.

Policy	Low response	High response	Low completion	High completion
Priority	59	0	59	59
Aging	49	0	49	66
Round Robin	0	1	2	63

All measurements are in ticks. Each row represents one execution of agingtest 1000 with CPUS=1.

Policy	Average response	Average turnaround
Priority	29.5	59.0
Aging	24.5	57.5
Round Robin	0.5	32.5

With aging, the low-priority job started at tick 49 while the high-priority job continued until tick 66. With strict priority, the low-priority job waited until tick 59, when the high-priority work finished. Round Robin had the lowest averages here. Single-run measurements illustrate behavior; runtime variation prevents treating small differences as a general performance guarantee.

I learned that aging can reduce a low-priority job's wait while preserving preference for high base priorities. This test demonstrates service before the competing job finishes; it does not prove starvation freedom for every workload. The first 400-chunk runs varied in duration, and one finished before the 49-tick threshold. I increased the high-priority work to 1000 chunks for all policies. The existing matmul ignores workload arguments, so agingtest uses an explicit loop count. Files changed: kernel/param.h, kernel/proc.c, user/agingtest.c, and Makefile.

```
xv6 kernel is booting
init: starting sh
$ agingtest
CPU workload: low=3 chunks, high=400 chunks
priority response turnaround (ticks)
0          49          49
49         0          52
Average response: 24.5 ticks
Average turnaround: 50.5 ticks
$
```

```
xv6 kernel is booting
init: starting sh
$ agingtest 1000
CPU workload: low=3 chunks, high=1000 chunks
priority response turnaround (ticks)
0          49          49
49         0          66
Average response: 24.5 ticks
Average turnaround: 57.5 ticks
$
```

```
xv6 kernel is booting
init: starting sh
$ agingtest
CPU workload: low=3 chunks, high=400 chunks
priority response turnaround (ticks)
49      0      26
0       27      27
Average response: 13.5 ticks
Average turnaround: 26.5 ticks
$ agingtest 1000
CPU workload: low=3 chunks, high=1000 chunks
priority response turnaround (ticks)
49      0      59
0       59      59
Average response: 29.5 ticks
Average turnaround: 59.0 ticks
$
```

```
xv6 kernel is booting
init: starting sh
$ agingtest 1000
CPU workload: low=3 chunks, high=1000 chunks
priority response turnaround (ticks)
0       0       2
49      1      63
Average response: 0.5 ticks
Average turnaround: 32.5 ticks
$
```

Task 6. (5 pts) Summary

This assignment helped me connect process states, system calls, and scheduling. A user program cannot read the process table directly, so `getprocs()` collects protected kernel information and copies it into user memory. Adding priority and readytime showed how process fields support both observation through `ps` and scheduling decisions.

I learned the difference between base and effective priority. Strict priority favors the highest base value and can keep a lower-priority process waiting. Aging raises a waiting process's effective value without changing its requested priority. Round Robin gave faster response for the short job tested here.

Obstacles included adapting to this xv6 version's five-argument `copyout()`, sharing `procstate` without duplicate definitions, preserving scheduler interrupt handling, and keeping `Makefile` valid. Source inspection, small commits, compilation, and tests helped check each stage. When QEMU was difficult to exit, I returned to Ubuntu and terminated it without deleting source files. The implementation and tests were committed on pa2. At report preparation time, the first push failed because the VM could not resolve `github.com`. The online upload and Teams submission remain separate steps to complete.

Extra Credit Task (10 points).

I implemented `getpriority` and `setpriority` as built-in commands in `user/sh.c`. The shell handles these commands before creating a child process. `getpriority` displays the current shell's base priority, while `setpriority` validates an integer from 0 to 49 and calls the corresponding system call. Invalid arguments produce a usage message. Because `setpriority` changes the shell itself, future child processes inherit the new priority through `fork()`.

To demonstrate the effect on response time, I ran `intertest 1000` with the aging scheduler and one CPU. The test releases two jobs through pipes: a short job representing interactive work and a longer CPU-intensive job with priority 30. The short job uses the priority inherited from

the shell. Response time is measured from the common release time until each job begins its CPU workload.

After `setpriority 0`, `getpriority` returned 0, and the short job's response time was 30 ticks. After `setpriority 49`, `getpriority` returned 49, and the same short workload had a response time of 0 ticks, meaning it started within the release tick. The longer job's response time changed from 0 to 1 tick. These measurements show how increasing the priority of short interactive work can reduce its waiting time when it competes with CPU-intensive work. This was a controlled demonstration using a short workload as a model of an interactive job.

```
xv6 kernel is booting
init: starting sh
$ setpriority 0
$ getpriority
0
$ intertest 1000
CPU workload: low=3 chunks, high=1000 chunks
priority  response  turnaround (ticks)
0          30        30
30         0        57
Average response: 15.0 ticks
Average turnaround: 43.5 ticks
$ setpriority 49
$ getpriority
49
$ intertest 1000
CPU workload: low=3 chunks, high=1000 chunks
priority  response  turnaround (ticks)
49         0         0
30         1        79
Average response: 0.5 ticks
Average turnaround: 39.5 ticks
$ setpriority 0
$ QEMU: Terminated
```

Use of AI assistance

I used OpenAI ChatGPT/Codex when I had questions about the xv6 tasks or encountered errors in my code. It helped me understand compiler messages and possible corrections, which I reviewed and tested in my Ubuntu VM.